Slide 1: Introduction to Matplotlib

🎯 **Purpose (1 line)**
Understand the ecosystem, components, and purpose of Matplotlib.

📌 **Key Idea (3–5 bullets)**
- Standard for static, publication-quality 2D charting in Python.
- Highly flexible, extensible, and widely adopted in data science.
- Supports multiple backends (screen, file output) and interactivity.
- Integrates seamlessly with NumPy, Pandas, and other Python tools.

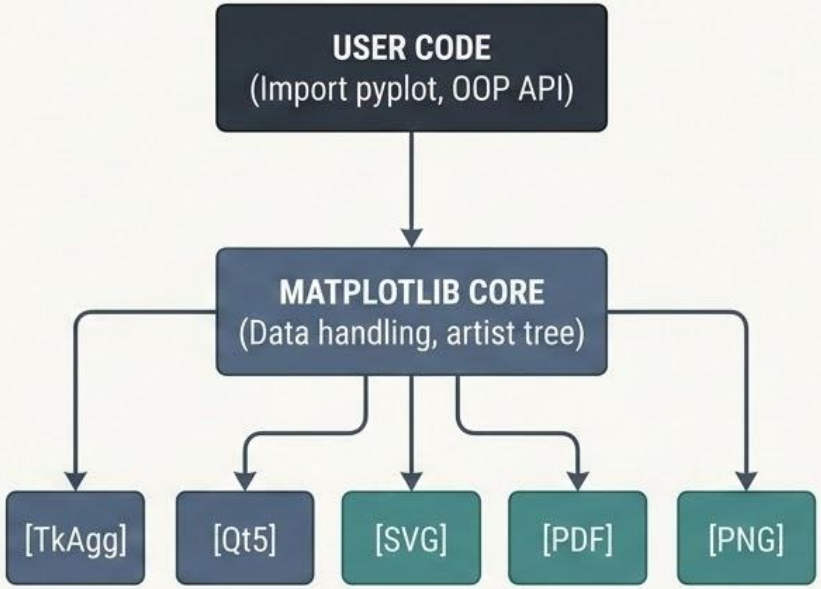
💻 **Minimal code**

```
import matplotlib.pyplot as plt
import numpy as np
```

* `plt`: Standard convention to import the plotting engine.
* `np`: Necessary for generating efficient numerical data.

📈 **Expected Output:** Expected Output
🧠 **Memory Trick:** Understand the caast lor memory trick
⚠️ **Common Mistakes:** Mistabs are mearly as common mistakes
✅ **Best Practice:** Prefines aninformay of best practice
🎓 **Interview Question:** What leat say about question ot Matplotlib?
🚀 **Challenge:** What errots this and can promarts challenge.

🖼️ **Visual Diagram: The Matplotlib Ecosystem**



Slide 2: Mastering `plt.figure()` and `plt.subplots()`

🎯 **Purpose (1 line)**
Understand Figure and Axes objects for optimal graph structuring.

🛠️ **Key Ideas (Bullets)**
- **Figure**: The overall window/page container (the canvas).
- **Axes**: The individual plotting area with a coordinate system (the graph itself).
- **`plt.subplots()`**: The recommended modern function to manage both.

💻 **Minimal Python Code (Syntax Highlighted):**

```
import matplotlib.pyplot as plt

# Explicitly create Figure and Axes
fig, ax = plt.subplots(figsize=(10, 5))
```

- Create objects simultaneously using the optimal OOP approach.

🔍 **Expected Graph Appearance:** One wide, empty plotting window.
🚀 **Real-world Use Case:** Comparing quarterly revenue data for three separater regions on distinct subplots one figure.

📝 **Parameters & Return (Table)**

Parameter	Description	Default
figsize	Tuple (W, H) in inches	(6.4, 4.8)
nrows	Number of rows	1
ncols	Number of columns	1

***Return:** `Figure`, `numpy.ndarray` of Axes.

✅ **Best Practice:** Use `plt.subplots()` instead of `plt.figure()`, then `plt.subplot()`.
⚠️ **Common Mistake:** Confusing the Figure (canvas) with the Axis/Axes (plot area).
💡 **Pro Tip:** Set `figsize` during creation; call `fig.tight_layout()` for complex grids.

🧠 **Memory Trick:** **F**igures are **A**rt canvases. **A**xes are **A**ctual graph carts.
🎓 **Interview Question:** What is the fundamental Matplotlib object hierarchy, and which method is preferred for multi-plot layouts?
🏋️ **Mini Exercise:** Modify the minimal code to create a 3x1 (rowsxcols) grid.

Types of Charts in Matplotlib

What charts are

Data Visualizations
Converting abstract numbers into understandable images.

Visual Storytelling
Communicating insights effectively to an audience.

Mathematical Graphics
Plotting mathematical functions and relationships.

Why different charts exist

Highlighting Relationships
Some charts show correlations, others distributions.

Serving Goals
Different visual encoding for comparison, part-to-whole, or trends.

Human Perception
Choosing the most readable method for the intended data type.

Adapting Data
Fitting data volume and complexity (e.g., small datasets vs. massive scale).

Where charts are used

Business Analytics
Dashboards, progress tracking, tracking, KPIs.

Scientific Research
Data exploration, pattern recognition, publication.

Financial Modeling
Stock performance, risk analysis.

Machine Learning
EDA (Exploratory Data Analysis), model evaluation.

Journalism & Media
News infographics, data journalism.

Revision: Line Plots in Matplotlib

What It Is & Key Styles

Visualize continuous changes over time or an interval.

Common Line Styles:

[-] ——— ———
[--] - - - - - - - -
[-.] —·—·— —·—·—
[:] ....

Common Markers:

[●] [■] [▲]

Anatomy: Function & Parameters

`plot()'

- [x]
- [y]
- [label]: Define line label
- [color]: Define line color
- [linewidth]: Define line linewidth
- [linestyle]: Define line style
- [marker]: Define line marker
- [markersize]: Define line markersize
- [alpha]: Define size and alpha

Best Practices

- Prioritize clarity and readability.
- Use meaningful labels and a title.
- Keep comparisons limited and effective.

Where It Is Used

Sales Trend
Stock Price
Temperature
Website Traffic
Time Series

Quick Example

```
plt.plot(dates, values, color='teal',
         linestyle='-', linewidth=2.5,
         label='Trend')
```

Common Mistakes

- Overplotting (too many lines)
- Poor color contrast

Interview Tip

Explain *why* line plots are best for showing continuous change or time series.

Memory Trick

Think connecting dots on a timeline.

Remember:
Use Line Plot whenever the X-axis represents time or ordered data.

Revision: Bar Charts in Matplotlib

What It Is & Common Uses

Visualize and compare discrete categories.

Common Uses:

- Sales by Product;
- Students by Class;
- Revenue by Department;
- Population by Country

Anatomy: Function & Parameters

`bar()`

- [x]
- [height]
- [width]
- [color]
- [edgecolor]
- [label]
- [alpha]
- [align]

Best Practices

- Use meaningful labels.
- Keep categories limited.
- Start Y-axis at zero.
- Avoid excessive colors.
- Use horizontal bars for long labels.

Vertical vs Horizontal Bar

Vertical (`bar()`) | Horizontal (`barh()`)

Flip axes for better readability with long category labels.

Quick Example

```
plt.bar(cats, sales, color='teal',
        label='Q1 Sales')
```

Common Mistakes

- Not starting Y-axis at zero (misleading comparisons).
- Poor category ordering.

Interview Tip

Explain *why* bar charts excel at showing discrete comparisons vs histograms for continuous data.

Memory Trick

Think sorting items into separate labelled bins.

Remember:
Use Bar Chart whenever you compare categories.

Revision: Scatter Plot in Matplotlib

Anatomy: Function & Parameters

What It Is & Common Uses

Visualize and analyze relationships between two numerical variables.

Common Uses:

- Correlation
- Outlier Detection
- Machine Learning Feature Analysis
- EDA (Exploratory Data Analysis)

`scatter()`

- **x**: (Required) Data for x-axis.
- **y**: (Required) Data for y-axis.
- **color/c**: Single color or array (for multi-dimensional analysis).
- **s**: Marker size (float or array).
- **marker**: Marker style (string, e.g., 'o', 's').
- **alpha**: Transparency level (0-1).
- **cmap**: Colormap for continuous `c` data.
- **linewidths**: Edge thickness of markers.

Different Marker Styles

'o' Circle	's' Square	'^' Triangle_up
'+' Plus	'.' Point	'x' X cross

Quick Example

Adds visual dimension

```
plt.scatter(study_hours, exam_scores,
  color='purple', alpha=0.6, marker='o')
```

Reduces overplotting | Sets shape

Common Mistakes

- Misinterpreting correlation as causation.
- Overplotting without adjusting `alpha`.
- Not standardizing data for better comparisons.

Interview Tip

Explain how the `c` (color) and `s` (size) parameters are used to add multi-dimensional analysis, representing third and fourth continuous variables on a 2D plot.

Memory Trick

Think of ***Scatter*** (points scattered) for finding connections, whereas ***Bars*** are for comparing discrete

Remember:

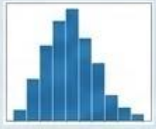
Scatter Plot is the best chart for finding relationships between variables.

Revision: Histogram in Matplotlib

Anatomy: Function & Parameters

`hist()'

```
plt.hist(data, bins=30,
    color='blue',
    edgecolor='white',
    alpha=0.7)
```



Separates bars

What It Is & Common Uses

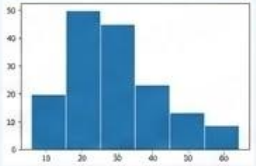
Visualize frequency and spread of continuous numerical data.

Common Uses:

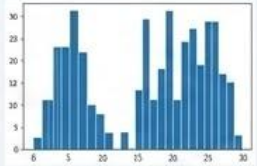
- Age
- Exam Scores
- Salary
- Data
- Heights
- Distribution
- Always label axes.

- a. **Best Practices**
- a. Choose optimal bins
- b. Confusion with Bar Charts (comparing categories).
- c. Setting too many/too few bins.
- d. Missing axis labels and titles.

Different Bin Sizes



Few Bins (oversmoothing)



Many Bins (overfitting)

Quick Example

Controls granularity

```
plt.hist(data, bins=30, color='blue',
  color='edgecolor='white', alpha=0.7)
```

Separates bars Sets shape

Common Mistakes

- Choose optimal bins based on data size.
- Use `**edgecolor**` for clarity.
- Check `**density=True**` for comparisons.
- Always label axes.

Interview Tip

Explain how `**bins**` impact visual interpretation and the difference between density and frequency plots.

Memory Trick

Think of ***Hist***ograms capturing a *hist*ory of counts across ranges, while ***Bars*** compare discrete *cat*egories.

Remember:

Histogram shows distribution, not category comparison.

Revision: Pie Chart in Matplotlib

Anatomy: Function & Parameters

What It Is & Common Uses

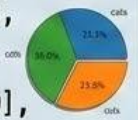
`pie()'

Visualize the relative proportions of categories within a whole.

Common Uses:

- Market Share
- Budget Allocation
- Population Percentage
- Survey Results
- Expense Breakdown

```
plt.pie(data, labels=cats,
    colors=cols,
    autopct='%1.1f%%',
    explode=[0.1, 0, 0, 0],
    shadow=True,
    startangle=90, radius=1)
```



- a. **Best Practices**
- a. Choose optimal bins
- b. Confusion with Bar Charts (comparing categories).
- c. Setting too many/too few bins.
- d. Missing axis labels and titles.

Most Important Parameters

- **x** - Dentiine tlices of data
- **labels** - Use the proportion categories
- **colors** - Control colons of colors
- **explode** - Compare optioct of explode
- **autopct** - Control bind of autopct
- **shadow** - Rescriptiiess of shadow
- **startangle** - Startate area of radius

Quick Example

Controls granularity

```
plt.pie(data, explode='%.1%',
        autopct=[0, 0, 0]=1)
```

Separates slices

Sets shape

Common Mistakes

- Showing parts of different wholes
- Using too many categories (hard to
- Using too many categories (hard to read)
- Missing labels or a descriptive title

Interview Tip

Explain when to use a Pie Chart vs. Bar Chart and how `**autopct**` and `**explode**` improve readability.

Memory Trick

Think of a **Pie Chart** capturing portions of a pie recipe, while **Bar Charts** compare items across recipes.

Remember:

Use Pie Chart **ONLY** when showing parts of a whole (100%). Avoid using Pie Charts with too many categories.

Matplotlib Charts – Final Revision Cheat Sheet

Which Chart Should I Use?

- Compare Categories → **Bar Chart**
- Compare Rankings → **Horizontal Bar Chart**
- Show Trend Over Time → **Line Plot**
- Find Relationship → **Scatter Plot**
- Show Distribution → **Histogram**
- Find Outliers → **Box Plot**
- Compare Distributions → **Violin Plot**
- Show Percentage → **Pie Chart**
- Show Composition Over Time → **Stack Plot**
- Visualize Images → **Imshow**
- Matrix Data → **Matshow**
- Vector Field → **Quiver**
- Flow Field → **Stream Plot**
- Surface/Density → **Contour**
- Large Scatter Data → **Hexbin**
- 3D Data → **3D Plot**

Top 7 Charts You Must Master

- ★★★★★ Line Plot
- ★★★★★ Bar Chart
- ★★★★★ Scatter Plot
- ★★★★★ Histogram
- ★★★★☆ Box Plot
- ★★★★☆ Horizontal Bar
- ★★★☆☆ Pie Chart

Most Common Customization Properties

- color
- label
- linewidth
- linestyle
- marker
- markersize
- alpha
- edgecolor
- figsize
- title
- xlabel
- ylabel
- legend
- grid

Interview Tips

- Know when to use each chart.
- Know the difference between **Bar** vs **Histogram**.
- Know **Scatter** vs **Line**.
- Know **Figure** vs **Axes**.
- Know `pyplot` vs **Object-Oriented API**.
- Know how to customize plots professionally.

Learning Roadmap

- Charts
- Customization
- Subplots
- Layouts
- Annotations
- Styles
- Saving Figures
- Advanced Plots
- Animations
- Publication Quality Graphs

Final Takeaway

'Choosing the correct chart is more important than creating a beautiful chart.'

'A good visualization tells the story with the least amount of effort.'